A heap machine for Ochr

The design behind the proposed port of the checker, what it proves, where it fails, and whether to port

Decision document · 4 November 2026 · written by the agent that proposed the design

Summary. The stability conjecture, that a definition's generic run and its runs at instances take the same decisions, rests on a lemma (P): a run that never looks at an unknown cannot depend on it. Against today's checker (P) is a step-by-step comparison of two runs through every function of a 116-function evaluator, estimated at 10,000 to 20,000 lines of Lean. The *heap design* makes (P) one lemma: values become handles into a heap, the machine can look at a handle only by an explicit request, and the machine is written as a data structure of requests, so "this run never asked about σ" is a fact about that data structure and the lemma is proved once, by induction on it, for every program at once. That lemma, the frame property and part of the correctness of closing off are proved in Lean on a slice of the machine, and the slice agrees with today's checker on every one of 1,166 runs of suite functions. Porting the checker to the design would take an estimated 4 to 8 weeks. This document argues the key idea, separates what is proved from what is measured or assumed, and gives the three referees' objections full weight. Their objections hold, and preparing this document found more. The design does not yet cover borrows returned by stuck calls; its references are not yet exclusive borrows; the treatment of unknowns' types it proposed would rebuild a known proof of `False`; it makes (P) free but not the steps that consult an unknown's *type* (copy or move, conversion); and (P) needs a renaming of fresh handles once a refinement allocates. Above all there is no bridge: after a port the checker would no longer implement the paper's rules, which are about moving borrows and resolution, and nothing proves the two machines equivalent. The recommendation is **not to port now**: first settle the design questions on paper and decide whether the paper should adopt the heap machine as Ochr's semantics; port if and only if it does.

Contents

1 The decision 3
2 The problem the design solves 3
2.1 Stability, and the lemma it rests on 3
2.2 Why (P) is expensive on today's machine 3
3 The design 4
3.1 Handles and nodes 4
3.2 Requests, computations and the interpreter 4
3.3 Refinement fills a node 5
3.4 Borrows are references to places 5
3.5 Closing off is a thunk 5
3.6 Capabilities 5
4 The key idea: why reification makes (P) one lemma 6
4.1 Runs agree where the heaps agree on what is asked 6
4.2 What follows from it 6
4.3 What it does not say 7
5 What changes from today's machine 8
6 What is proved, measured and assumed 9
7 Where the design fails or needs redesign 9
7.1 Whole-value walks 10
7.2 Tables keyed by contents 10
7.3 The type of an unknown, and dependent fields 10

7.4 Steps that consult an unknown's type 11
7.5 Renaming of fresh handles 12
7.6 Locality is a hypothesis 12
7.7 References are not yet exclusive borrows 12
7.8 Borrows returned by stuck calls 12
7.9 Performance 13
8 The bridge: two machines 13
9 Costs and an order of work 14
10 Recommendation 14
11 Open design questions, collected 15

1 The decision

The question for the user is whether to rewrite the value layer of Ochr's checker around a heap, a rewrite of the core of a machine of about 3,500 lines that I estimated at 4 to 8 weeks of focused agent work to reach agreement on every one of the suite's verdicts. I proposed it, in the stability work of October 2026, as the way to make one lemma of the consistency plan provable at reasonable cost. Since then four lanes have extended the design's slice and its proofs, and three referees attacked the consistency plan it belongs to (docs/18), several of them on this design specifically.

This document states the design precisely (Section 3), argues the one idea that makes it worth anything (Section 4), says what it removes from today's machine (Section 5), lists what is proved, what is only measured and what is assumed (Section 6), and then looks for where it fails, giving the referees' points full weight (Section 7, Section 8). It closes with costs, an order of work and a recommendation (Section 9, Section 10). It stands on its own: where a result is proved in Lean, the file and declaration are named, but every argument is made here.

> **Quick explainer: what "the two paths" are.** Ochr checks a definition once, at its *generic call*: each parameter is bound to an *unknown* σ (an abstract value of the parameter's type), and the body is run. Where the run needs to look inside an unknown, it either *splits* (a `match` on σ is checked once per constructor, with σ refined to that constructor in each arm) or *closes off* (a call stuck on σ returns a sealed program, $\lceil t \rceil$, that stands for the call and re-runs when σ becomes known). A proof checked this way is then used at every *instance*, where the unknowns are actual values and the same code simply runs. If the generic run and an instance run ever decide something differently, a statement proved generically can be false at the instance, which is a closed proof of `False`. *Stability* is the conjecture that they never do.

2 The problem the design solves

2.1 Stability, and the lemma it rests on

A *refinement* α assigns values to unknowns, $\sigma \mapsto v$. Stability says: for a run of a term t from a state Ω on the generic path, refining afterwards gives what running the refined term from the refined state gives,

$$(\mathsf{run}(t, \Omega))\alpha \quad \text{agrees with} \quad \mathsf{run}(t\alpha, \Omega\alpha),$$

where "agrees" is equality up to the renaming of fresh names and the ending of borrows, and where every decision the checker takes along the way (whether a term is erased, a function type's class, how a stuck block captures its places, which places an observation reads) comes out the same. The stability work split the run half of this into three lemmas, written N = (P) + (C) + (Sp), plus a fourth, (M), that typed and untyped runs agree:

- **(P) parametricity.** A run that never looks at σ (never matches on it, compares it, or reads through it) computes the same steps with $\alpha(\sigma)$ in its place.
- **(C) closing off is correct.** Re-running a sealed program under α gives what the direct call gives at the instance.
- **(Sp) splits commute.** Where the generic run splits σ, the arm for $\alpha(\sigma)$'s constructor, refined further, is the instance's run.

(P) is the base: (C) reduces to it plus the frame property (a call affects only what it is passed), and (Sp) to the rules for what each arm of a split may see and keep.

2.2 Why (P) is expensive on today's machine

In today's machine a value is a tree, an unknown σ is a leaf `abs σ` inside it, and a refinement is a *substitution*: it replaces σ throughout the environment, every stored type, the types of the other unknowns, the goal, and the code inside every sealed program, and then re-runs each sealed program that changed. To prove (P) one must show that substituting before a run and substituting after it give the same result, which means following both runs side by side through every step of the machine.

That was measured, not guessed. A lemma about the real substitution, that on first-order data it has no effect on the state and computes a structural substitution, is **proved in Lean against the checker** (`PProbe.substV_data`, `Scratch/PProbe.lean`, about 40 lines): the machine's monad and its fuel-indexed functions are not in themselves an obstacle. But unfolding the real `eval` on the numeral `0`, the simplest term there is, produces a proof state of about 50,000 characters, almost all of it bookkeeping: the erasure pre-pass, the flags that record whether a term was a proof, the typing and erasure depths, the editor's source locations, each of which needs its own invariant before a single case of (P) closes. The machine's evaluator is a cycle of 116 mutually recursive functions over a state of about 30 components. A direct proof needs a relation between the two runs' states preserved by every one of those functions; the estimate recorded with the measurement is 10,000 to 20,000 lines of Lean, and fragile under any change to the checker.

Why: (P) is a non-interference *property: what a run does cannot depend on something it never looked at. Such properties are hard to prove when looking is implicit, as it is when a value is a tree the code can pattern-match on anywhere, and easy when looking is an explicit, visible act. The design makes it visible.*

3 The design

3.1 Handles and nodes

A value is a *handle*, a number naming a node of a *heap*. A heap H maps handles to nodes and has an allocation counter $\mathsf{next}(H)$, the next fresh handle. A node is one of:

- a constructor applied to the handles of its fields, $\mathsf{C}(a_1, ..., a_k)$, including Z, $\mathsf{S}(a)$ and `()`;
- a *cell* $\mathsf{cell}(v)$: a mutable place, such as a variable's binding, holding the handle v of its current value;
- a *reference* $\mathsf{ref}(c, \pi)$: a borrow, pointing at the place π (a path of fields) inside the value held by cell c;
- a *thunk* $\mathsf{thunk}(f, \overline{x})$: a delayed call of function f on the input handles $\overline{x}$, which is how a stuck call is closed off (Section 3.5);
- an *opaque* node holding a value outside the data the slice covers (a proof $\star$, a function, a type).

An *unknown* is a handle with no node: $H(\sigma)$ is empty. It is not a special kind of value; it is the absence of an answer.

3.2 Requests, computations and the interpreter

The machine never reads the heap directly. It is written as a *reified computation*: a tree whose internal nodes are requests to the heap, each followed by the rest of the computation as a function of the answer. There are three requests:

- $\mathsf{view}(h)$ asks what h holds and continues with the answer, a node or nothing (for an unknown);
- $\mathsf{alloc}(n)$ stores the node n at a fresh handle and continues with that handle;
- $\mathsf{write}(h, n)$ overwrites the node at h, for assignment, and continues.

A leaf $\mathsf{ret}(b)$ ends the computation with the result b. In Lean this is the inductive type `Prog` (`Ochr/Heap/Slice.lean`), with constructors `ret`, `view`, `alloc` and `write`.

An *interpreter* runs a computation p against a heap H, answering each request from the heap, and returns the outcome $\mathsf{out}(p, H)$ and the final heap $\mathsf{heap}(p, H)$ (Lean: `exec`). Alongside it, $\mathsf{asked}(p, H)$ is the list of handles the run views or writes, in order (Lean: `queries`). Both are defined by recursion on the tree, one clause per request.

> **Quick explainer: reified computations and their interpreters.** A *reified computation* is a program represented as data. Instead of a function that reads and writes a heap, it is a tree: each node is a request ("what does handle 7 hold?", "store this node"), and its children are the rest of the program, one for each possible answer. A separate *interpreter* walks the tree and answers each request against an actual heap. This is the "free monad" pattern of functional programming. Because the program is data, properties of "every program" can be proved by induction on trees, once; and because every access to the heap is a visible node, "this run never asked about σ" is something one can compute and reason about. Nothing is lost in expressiveness: the machine is still an ordinary recursive function, which happens to build the tree as it goes, and running it is interpreting the tree as it is built.

3.3 Refinement fills a node

A refinement does not substitute. It fills the unknown's node in place: $H[\sigma := n]$ is H with $H(\sigma) := n$, every other handle and the counter unchanged. Every value, type and delayed call that mentions σ holds the handle σ, so all of them see the refinement at once, with no traversal: the environment, the stored types and the thunks are not touched. A case split on σ enters each arm by filling σ with that arm's constructor applied to fresh unknowns, and leaves the arm by emptying σ again.

3.4 Borrows are references to places

Today a borrow $\&p$ moves the content v of p into a value $\mathrm{borrow}_\ell v$ and leaves a *loan* loan_ℓ at p; ending the borrow puts the (possibly changed) content back by substituting it for the loan wherever it occurs, and observing a statement first ends every borrow, which is *resolution*. In the heap design, $\&p$ allocates a reference $\mathsf{ref}(c, \pi)$ to the place: $*r$ follows the reference to the cell, and $*r := t$ writes the cell, rebuilding the value along the path. The borrowed place always holds its current content, so there is nothing to put back, and the ending of a borrow, resolution and the search for a loan's owners have no counterpart.

Calls pass borrows *by copy and restore*: at the call, each borrowed place's content is copied into a fresh cell, the callee borrows that cell, and when the callee returns, the cell's final content is written back to the place. Because borrows are exclusive and never stored in data, this is the same as passing the reference itself; it is chosen because it keeps the callee away from the caller's cells (Section 3.6), and because a callee that gets stuck has run only on fresh cells, so discarding its partial run leaves the caller untouched.

> **Quick explainer: copy and restore.** A calling convention, also called *call by value-result*: the callee gets a private copy of each argument passed by reference, and its final value is copied back when the call returns. It is observably the same as passing a reference when nothing else can see the place during the call, which in Ochr is guaranteed because a borrow is exclusive: while a place is lent, nothing else may use it.

3.5 Closing off is a thunk

When a call gets stuck on an unknown, its partial run is discarded and the call becomes thunks over its inputs: one thunk stands for its result, and one for the final content of each borrowed place, which is written into that place. A call of a function returning `Unit` returns `()` instead of a result thunk, as today. Where a run looks at a value that is a thunk, the thunk is *forced*: the delayed call is run on its inputs as the heap now holds them, refinements included, without closing off its own head call again (today's [Seal] head guard). If the forced call returns, the thunk's node is replaced in place by the outcome's node; if it is still stuck, the thunk stays. Forcing is today's re-normalisation of a sealed program, done when the value is looked at rather than whenever a refinement happens.

3.6 Capabilities

A *capability interpreter* runs a computation holding a set R of handles it may use: it may view or write only a handle in R, and store only nodes whose children are in R; viewing a node adds the node's children to R, and allocating adds the fresh handle. Any other request is a failure. Written $\mathsf{execR}(p, H, R)$ (Lean: `execR`, `Ochr/Heap/Frame.lean`). $\mathsf{Reach}_H(R)$ is the set of handles reachable in H from R, and $H \restriction R$ is H with every node outside $\mathsf{Reach}_H(R)$ removed: the heap a call would see if it ran alone.

> **Quick explainer: capabilities.** An *object-capability* discipline lets code act only on references it was given or created, never on references it could compute or guess. A callee given handles $\overline{x}$ can then affect only what $\overline{x}$ reaches, and the caller's other state is out of its reach by construction rather than by a proof about the callee. It is the operational form of the *frame property* of separation logic.

4 The key idea: why reification makes (P) one lemma

4.1 Runs agree where the heaps agree on what is asked

Lemma 1 (runs agree on what they ask). Let H_1 and H_2 be heaps with the same counter, $\mathsf{next}(H_1) = \mathsf{next}(H_2)$, and let D be a set of handles below that counter such that $H_1 \simeq_D H_2$: the two heaps hold the same node at every handle outside D. If the run of a computation p from H_1 asks about no handle in D, that is $\mathsf{asked}(p, H_1) \cap D = \emptyset$, then $\mathsf{out}(p, H_1) = \mathsf{out}(p, H_2)$, the final counters are equal, and $\mathsf{heap}(p, H_1) \simeq_D \mathsf{heap}(p, H_2)$.

Proof sketch. By induction on the tree p, with H_1 and H_2 general. A leaf $\mathsf{ret}(b)$ returns b from both and changes nothing. A request $\mathsf{view}(h)$: h is asked, so $h \notin D$, so $H_1(h) = H_2(h)$; both runs receive the same answer and continue with the same subtree, whose asked handles are the rest of the list, and the induction hypothesis applies. A request $\mathsf{alloc}(n)$: the counters are equal, so both runs store n at the same fresh handle and continue with the same subtree; that handle is not in D, because D lies below the counter, so the heaps still agree outside D, and both counters move up by one. A request $\mathsf{write}(h, n)$: both heaps get n at h, which keeps them in agreement outside D (and at h itself). □

It is `Ochr.Heap.exec_agree`, **proved in Lean, heap slice, main**, axioms `propext` and `Quot.sound`.

> *Why: The lemma never mentions the machine. It quantifies over every computation tree, so it holds for any machine written as one: the induction is over the four kinds of request, not over the machine's constructs. That is where the 10,000 to 20,000 lines go: they were the per-construct cases of a proof about a machine that reads its state implicitly, and here there are no per-construct cases, because the only code that reads the heap is the interpreter.*

Why this needs reification, and not just polymorphism. An obvious alternative is to keep the machine an ordinary function and make it polymorphic in the type of unknowns, so that it cannot inspect one. Then (P) would be *true*, as a free theorem in Reynolds' sense. But Lean's logic has no parametricity principle: a free theorem about a particular function has to be proved by induction over that function, one case per construct, which is the per-function proof that was too expensive. Reification moves the burden from the program to the data structure the program builds. The interpreter is the only thing that consults the heap, and the one induction needed is over the interpreter's four cases. A further gain: "this run never looked at σ" stops being a semantic side condition and becomes a computable fact, $\sigma \notin \mathsf{asked}(p, H)$, which can be checked on any run.

4.2 What follows from it

Corollary 2 ((P) on the slice). Let σ be a handle below $\mathsf{next}(H)$ and n a node. If the slice evaluator's run of a term t in an environment ρ never asks about σ in H, then its outcome is the same in $H[\sigma := n]$, and its final heap is the same except at σ.

Proof sketch. Lemma 1 with $D = \{\sigma\}$: H and $H[\sigma := n]$ have the same counter and agree everywhere except σ.□

It is `Ochr.Heap.P_evalH`, **proved in Lean, heap slice, main**. No case of the evaluator is inspected.

Lemma 3 (locality and the frame property). (a) A capability run that succeeds asks only about handles reachable from what it held, or fresh ones: if $\mathsf{execR}(p, H, R)$ succeeds then every handle in $\mathsf{asked}(p, H)$ is in $\mathsf{Reach}_H(R)$ or at least $\mathsf{next}(H)$. (b) If $\mathsf{execR}(p, H, R)$ succeeds, its outcome is the outcome of the isolated run from $H \restriction R$; on the handles reachable from R and the fresh ones, its final heap is the isolated run's final heap; and every other handle keeps the node it had in H. (c) A callee run from the handles of its arguments, through a borrow argument, reaches the cell it was given and nothing else of the caller.

Proof sketch. (a) by induction on the tree: a view of a held handle grants only that node's children, which were reachable already; an allocation grants the fresh handle. (b) by (a) and Lemma 1, with D the handles that are neither reachable from R nor fresh: H and $H \restriction R$ agree outside D and have the same counter, and by (a) the run asks about nothing in D. A handle the run never asks about keeps its node. (c) is (b) applied to a call. □

They are `cap_queries`, `untouched`, `frame` and `call_effect` (`Ochr/Heap/Frame.lean`), **proved in Lean, heap slice, main**, axioms `propext`, `Quot.sound` and `Classical.choice`.

Lemma 4 (forcing a thunk is the direct call). Let a thunk at h in heap H_f delay the call of f on inputs $\overline{x}$, and let the direct call's capability run from $\overline{x}$ succeed in a heap H_d. If H_f and H_d are well formed (no node

at or above the counter), have the same counter, and agree on every handle reachable from $\overline{x}$, then forcing the thunk in H_f takes exactly the direct call's outcome as the thunk's value.

Proof sketch. Forcing runs the delayed call on $\overline{x}$ in H_f. By Lemma 3 (a) the direct run from H_d asks only about what $\overline{x}$ reaches and fresh handles; the two heaps agree on the first and, being well formed with equal counters, on the second; Lemma 1 gives the same outcome. □

It is `force_is_direct`, with `delayed_agree` and `delayed_hole` (the last is Corollary 2 for a hole), in `Ochr/Heap/Close.lean`: **proved in Lean, heap slice, branch `heap-close`**, unmerged.

4.3 What it does not say

The referees were right that these results are weaker than their names suggest, and the limits matter for the decision:

- **They say nothing about the checker.** They are theorems about computation trees, so they apply to the checker only once the checker is written as one. Until the port, the lemmas cover the slice evaluator, and the slice's agreement with the checker is measured (Section 6), not proved.
- **(P) assumes the same counter.** Corollary 2 fills σ with one node whose children already exist. A refinement to a compound value allocates, and then the counters differ (Section 7.5).
- **The frame lemmas assume the run succeeded under capabilities.** That an evaluator only ever asks about handles it holds is locality itself; it is measured, not proved (Section 7.6).
- **Forcing assumes the inputs are not themselves thunks, and the same counter.** At an instance the inputs were computed directly; on the generic path they may be thunks. And forcing happens after other allocations, so the counters differ (Section 7.5).
- **Nothing here covers a run that does ask about σ.** Matching on σ, comparing it, and asking whether its type is a copy type are all outside (P) by design. The first two belong to (Sp) and (C); the last was not covered at all (Section 7.3).

5 What changes from today's machine

Today	Heap design	Consequence
a value is a tree; an unknown is a leaf `abs σ`	a value is a handle; an unknown is a handle with no node	nodes have identity and can be shared
refinement substitutes through the environment, stored types, unknowns' types, goal and sealed programs, then renormalises	refinement fills one node	substitution (`substV`, `substT`, `substEnv`, `renormAll`, 19 uses of `substV` alone) is deleted
a borrow is $\mathbf{borrow}_\ell v$, with $\mathbf{loan}_\ell$ left at the place; ending it substitutes the content back	a borrow is a reference to the place	ending a borrow, resolution and the search for owners go; a lent mark per place must stay (Section 7.7)
a call moves its borrow arguments into the callee	copy and restore through fresh cells	a callee cannot reach the caller's cells; a stuck callee leaves no trace
closing off returns sealed programs $\lceil L; C \rceil$, with fills $\lceil L; C; c_i \rceil$ in lent places, renormalised eagerly on every refinement	closing off allocates thunks over the input handles, forced when looked at and replaced in place	forcing is a write that worlds must undo (Section 7.3); returned borrows with holes are not designed (Section 7.8)
a statement runs on a private copy of the state	a statement runs, then the heap is rolled back	needs a journal of writes; allocations can stay, unreachable
a split arm refines by substitution; leaving it restores the state except a keep list	an arm fills σ; leaving it empties σ and rolls back	the keep list becomes explicit: what survives a world is what is not rolled back
decisions (erasure, class, rows) from syntax and declared types	unchanged	already made where the machine cannot run (`Decide.lean`)
`Id` ends every borrow, then reads the footprint's final contents	`Id` reads the footprint's cells	owners are found by following references, not by searching

Figure 1: Today's machine against the heap design.

What does not change is the calculus as a user sees it: typing is the machine run with types, `Id` compares observations, a stuck call is closed off rather than rejected, erasure is decided from syntax. The design changes how values and borrows are represented, and with them which facts are visible to a proof.

6 What is proved, measured and assumed

Claim	Status	Where; what it assumes
runs agree on what they ask (Lemma 1)	**proved in Lean, heap slice, main**	`exec_agree`; generic in the computation
(P) on the slice (Corollary 2)	**proved in Lean, heap slice, main**	`P_evalH`; a fill by one node whose children exist, same counter
locality, frame (Lemma 3)	**proved in Lean, heap slice, main**	`cap_queries`, `untouched`, `frame`, `call_effect`; the frame part assumes the run succeeded under capabilities
the slice evaluator respects capabilities	**assumed: a hypothesis of a proved lemma**	measured: 0 refusals in 1,166 runs; a lemma "never fabricates a handle" is open
forcing is the direct call (Lemma 4)	**proved in Lean, heap slice, branch `heap-close`**	`force_is_direct`; assumes the same counter and inputs that are not thunks
substitution is pure on data, today's checker	**proved in Lean against the checker**	`PProbe.substV_data`
the slice agrees with today's checker	**measured, not proved**	98 functions (37 with a borrow parameter), 1,166 runs: 1,004 same value, 162 closed off on both, 0 differences; 469 final contents of borrowed cells compared
forced thunks equal direct calls	**measured, not proved**	579 of 579
(P) holds on today's checker	**measured, not proved**	193 of 193 runs where the heap run never asked about the unknown
whole-value walks defeat (P)	**measured, not proved**	first, 48-function slice: the unknown unasked in 22 of 94 runs with targeted reads, in 3 if every read walks
17 of 50 walk sites replaced by an index	**measured, not proved**	branch `port-step1`: checked equal to the old scans on all 1,828 verdicts
renaming invariance of fresh handles	**open**	needed by (C) and by every comparison with an instance
steps that consult an unknown's type	**open**	Section 7.3
borrows returned by stuck calls	**open**	no heap counterpart yet (Section 7.8)
the heap machine is today's machine	**measured, not proved**	the bridge (Section 8)
"a type per unknown, never changed by refinement"	**false as first stated**	rebuilds a known proof of `False` (Section 7.3)
"no renaming for the runs (P) covers"	**false as first stated**	not for compound refinements (Section 7.5)
"N = (P) + (C) + (Sp) is complete"	**false as first stated**	misses monotone steps (Section 7.3)

Figure 2: Status of every claim this document relies on.

What the measurements cover. The slice is the untyped machine on: variables, field and predecessor places, constructors, `let`, sequencing, matches on constructor values, `clone`, borrows and assignment through them, calls by copy and restore, and closing off as thunks. It does not cover: moves (every read copies), borrows returned by stuck calls, the typed path (so no case splits: a generic run that meets a `match` on an unknown is stuck, not split), conversion, dependent fields, or code that is generic in a type. The 1,166 runs are of functions the checker had already accepted, so a disagreement would have meant the representations differ, not that the heap design checks programs correctly: it is not a checker.

7 Where the design fails or needs redesign

I designed this, so this section looks hardest for where it fails. Eight problems follow, then a note on performance. The stability work itself found the first two (walks, and tables keyed by contents). The referees of docs/18 found the next three (the type of an unknown, the steps that consult a type, renaming) and pointed at the sixth (locality is a hypothesis), which the `frame-model` lane had recorded. The seventh (exclusivity) I found while preparing this document, and the eighth (returned borrows) both a referee and I. The bridge between the two machines, the largest, has its own section (Section 8).

7.1 Whole-value walks

A *walk* is a computation that visits every node of a value, or every position of the state, to answer a question such as "is any part of this value moved out?", "which loans occur in it?" or "who holds the borrow with this label?". On the heap design a walk views every node it passes, and a value that contains σ makes it view σ, even though the answer (σ holds no moved-out part and no loan) is the same on both paths. Viewing is asking, so Lemma 1′s hypothesis fails and (P) does not apply.

This is measured. On the first version of the slice (48 functions), with reads that look only where they go, the unknown was never asked about in 22 of 94 runs with one unknown; with every read walking the value it reads, as today's checker does when it checks for moved-out parts and searches for loans, in 3. So the walks have to go before (P) is worth anything.

Today's checker has 50 walk sites. Seventeen of them scan the whole state for the holder of a borrow or the liveness of a loan; branch `port-step1` replaced them with an index from borrow labels to the positions holding them, kept current by every function that writes a position, and checked equal to the old scans on every lookup made while checking the whole suite (**measured, not proved**, unmerged). The other 33 walk a single value: for moved-out parts (9 sites), for the loans in it, for owners, for `&` inside a type. They need a *summary on each node* (does anything below hold a moved-out part, which loans occur below), set when the node is allocated from its children's summaries.

Summaries fit the heap design and not today's machine, and the reason is worth stating. A summary computed at allocation records what lies below a node, including "nothing" below an unknown. It stays correct as long as filling an unknown never puts a moved-out part or a loan below it. A refinement satisfies that: it assigns *borrow-free* values (no borrow, no live loan), and it never assigns $\bot$. Forcing a thunk replaces a node with a call's outcome, and an outcome is whole (a function's result and a borrowed place's final content must be whole when they leave the call), so forcing satisfies it too. That argument is **argued here, not mechanised**. In today's machine, by contrast, a value is an immutable tree built at about 660 places in the checker, and adding a summary means touching all of them.

7.2 Tables keyed by contents

Some of the checker's tables are looked up by comparing values: the generalisation records (a sealed program n replaced by a fresh unknown, found again whenever n is re-derived), the stack of function pairs being compared by conversion, the cache of function types' classes, and about a dozen equality tests on values. A comparison by contents views both values, unknowns included, so each lookup is a question about every unknown in the key.

The fix is to key by node identity, which asks nothing. But identity is stricter than equal contents. Today, when the same sealed program is derived twice, the second copy is recognised by its contents and replaced by the same unknown (the checker's rule D34). With identity keys, two thunks for the same call built at different times are different handles, and a re-derivation is missed. Whether a miss is only an incompleteness (the second copy stays stuck as its own unknown) or reintroduces the failure D34 was made to fix has to be argued per table; it is **open** (H6). *Hash-consing* (allocating equal nodes as one) would restore recognition by contents, but it makes an allocation's answer depend on what the heap holds, behind σ's node included, so Lemma 1′s allocation case fails. It is ruled out unless the lookup is itself a request, which brings the questions back.

7.3 The type of an unknown, and dependent fields

The stability work listed, as a prerequisite of the port, that "every unknown carries a type, set when it is created and never changed by refinement", so that questions about σ's type need not view anything. A referee showed this is **false as first stated**, and that building it in would rebuild a closed proof of `False` the current checker accepts.

The counterexample. A field's type may depend on a value parameter. With

```
def G (n : Nat) (b : Bool) : Type := match n { Z => Void, S(m) => Nat }
inductive DP (n : Nat) := MkDP(x : Bool, y : G(n, x), z : Nat)
```

an unknown $\sigma_d : \mathsf{DP}(\sigma_n)$ has a field y whose type, $\mathsf{G}(\sigma_n, \sigma_x)$, is stuck generically, `Void` in the world where $\sigma_n := \mathsf{Z}$, and `Nat` in the world where $\sigma_n := \mathsf{S}(m)$. The type of the field's unknown must change with the world. Today's checker records, when it first takes σ_d apart in one world, field unknowns typed in that world, and keeps the record when the world ends, on the claim that it "follows from the type alone". In the sibling world it reuses them. The program `EtaBoom` (`Bad1`, `Boom : False`) exploits this and is accepted on main, with a control rejected; it is open, recorded in docs/17.

The corrected design, and a second trap. An unknown's type should be a *handle* to a type value, fixed when the unknown is created, while the type value itself lives in the heap and is refined like any other: the type of σ_y is a thunk for $\mathsf{G}(\sigma_n, \sigma_x)$, forced when looked at. That removes the cache. But forcing replaces the thunk in place, which is a write: forced in the world $\sigma_n := \mathsf{Z}$, the type node becomes `Void`, and if leaving the world does not undo that write, the sibling world sees `Void`: `EtaBoom` again, through memoisation instead of a cache. So leaving a world must roll back every write made inside it, forcings included, and anything deliberately kept across worlds (today's η-records) may keep only shapes whose types are still unforced. This is **argued here, not mechanised** and not implemented (H3, H4).

7.4 Steps that consult an unknown's type

N = (P) + (C) + (Sp) is **false as first stated** as a decomposition: some steps look at an unknown without getting stuck or splitting, and none of the three covers them:

- **copy or move.** Reading a variable $x : T$ copies its value if T is a copy type and moves it, leaving $\bot$, otherwise. Generically T may be an unknown type, not known to be a copy type, so x is moved; at an instance with $T := \mathsf{Unit}$ it is copied.
- **η at `Unit`.** A value whose type is known to be `Unit` is `()`; generically the type may not yet be known to be `Unit`.
- **conversion and `Eq`.** Two values may not be convertible generically and convertible at an instance, so an equation is stuck generically and `True` at the instance.

In each case the generic answer is the *less defined* one: a move where the instance copies, a value not yet normalised, a stuck equation. So the two paths agree not up to renaming but up to an ordering $\sqsubseteq$ in which $\bot$ lies below every value and a stuck equation below what it computes to. A referee showed the Lean statement of stability, which demands equality, is false for this reason at a reachable state of an accepted program, `G (n x : Nat) : Unit := ((match n { Z => (let y = x; ()), S _ => () }); ())`: the generic run's stuck block takes `x` by move, because one arm moves it, so the refined generic state holds `x = ⊥`, while the direct run at $n := 1$ keeps `x = 1`.

This is not a defect of the heap design in particular; it is a fact about Ochr that the decomposition missed. But it limits what the heap design buys. The slice copies on every read and has no type-generic code, so its (P) never met these steps. On the full design every read in type-generic code asks about its type's node to decide copy or move, so (P) applies to no run of a type-generic definition that reads a variable. The missing lemma, call it (Mono):

Conjecture 5 ((Mono) monotone questions). A run whose only questions about the unknowns in D are monotone ones (is this type a copy type, is it `Unit`, are these values convertible) agrees with the run on the refined heap up to $\sqsubseteq$: the generic outcome and final heap lie below the refined ones.

Can (Mono) be generic, as Lemma 1 is? Only in part. One can make a monotone question a request of its own kind, whose answer can only change from "no" to "yes" as unknowns are filled. A generic lemma would then need each *continuation* to be monotone in the answer: following "no" must lead to an outcome below following "yes". That is a property of the code at each such request (a move differs from a copy by a write of $\bot$, an unnormalised value from a normalised one, a stuck equation from a computed one), not of trees in general. So (Mono) is a lemma per request site. There are only a few kinds of site, which makes it far smaller than a per-function proof of (P), but it is not free. It is **open** my guess is one to two weeks after a port. (M), that typed and untyped runs agree, also holds only up to $\sqsubseteq$: typed runs decide copy or move from the stored type, untyped runs from the value's constructor.

7.5 Renaming of fresh handles

Corollary 2 compares H with $H[\sigma := n]$ at the same counter, where n is one node whose children already exist. A refinement to a compound value, such as $\sigma := \mathsf{S}(\mathsf{S}(\mathsf{Z}))$, must allocate the inner nodes, and an instance whose argument the caller built has those nodes elsewhere in the heap. Either way the counters differ, every fresh handle of the later run is shifted, and the outcomes agree only up to a renaming of fresh handles. The claim made with the slice, that "fresh names coincide, so runs (P) covers need no renaming", is **false as first stated** for such refinements.

Two remedies, which are not alternatives:

- **A refinement namespace** (**argued here, not mechanised**). If a refinement's nodes are allocated from a counter of their own, disjoint from the machine's, the machine's counter is unchanged and the refinement's nodes are reachable only through σ. Lemma 1 then applies with $D = \sigma$ and the refinement's nodes, because a run that never views σ never obtains a handle to them, given the capability discipline. That rescues (P).
- **Renaming invariance** (**open**). It is needed anyway. Forcing a thunk happens later than the call, after other allocations, so the forced run's fresh handles are the direct run's shifted; and relating an instance, where the caller built the arguments, to the generic heap filled in place is a renaming of where the argument's nodes live. Renaming invariance is not generic: a computation receives a fresh handle as a number and could branch on it. It is a property of the evaluator, that it only passes handles along. Written polymorphic in the handle type, the evaluator cannot inspect a handle, and the per-construct proof is mechanical. The `heap-close` lane estimated it at 2 to 3 weeks.

7.6 Locality is a hypothesis

The frame lemmas (Lemma 3 (b), (c)) hold for every run the capability interpreter accepts. That the evaluator's runs are accepted, that it never asks about a handle it did not get from its environment, a view or an allocation, is the locality of the evaluator, and it appears as a hypothesis. It is measured (no refusal in 1,166 runs), not proved. Two ways to discharge it:

- A lemma "the evaluator never fabricates a handle", per construct, estimated at about a week. Polymorphism in the handle type makes it true by construction, but, as for (P), Lean needs it proved case by case; the cases are mechanical.
- Run the checker under the capability interpreter, so that a violation is an internal error. Then every accepted program's *generic* run respected capabilities, by construction. That is not enough on its own: the proof also needs locality for the instance runs, which the checker never performs. So the lemma is still needed.

7.7 References are not yet exclusive borrows

Today exclusivity is enforced through the loan: while a place is borrowed it holds loan_ℓ, so reading or writing the place finds the loan and ends the borrow, after which using the borrow is an error. In the slice, $\&p$ allocates a reference and nothing records that p is lent. With plain references, `let r = &x; x := 1; *r := 2` writes `x` through a borrow that today's checker has already ended and rejects. The slice never meets this because it runs only programs the checker accepted; a heap *checker* must reject them.

So a heap checker needs a mark on each lent place, set by $\&p$, checked and cleared by every access to the place, which invalidates the reference. That is the loan again, without the substitution. The accurate statement of what the design removes is therefore "ending a borrow by substitution, and resolution", not "loans". Exclusivity is part of the bridge (Section 8) and of the port's cost; it is **open** (H5).

7.8 Borrows returned by stuck calls

The hardest feature of today's borrow machinery has no heap counterpart yet (**open**). A stuck call whose declared result is $\&T$ returns $\mathrm{borrow}_k\lceil ... \rceil$, and leaves in each borrowed place a fill containing a *hole* loan_k, which stands for whatever the caller will later write through the returned borrow. When the caller's borrow k ends, the hole is supplied by substitution. This is where resolution earns its keep, and the reason the paper's example `Pick` shows that the two paths agree only after every borrow has ended.

The natural heap form is a fresh cell for the hole: the returned reference points into it, and the fills are thunks that read it. But a thunk reads the heap when it is forced, and forcing is memoised. Forcing a fill before the caller has finished writing through the returned reference would read an intermediate content and freeze it. So the design needs to know when the hole is final, and that is exactly what ending a borrow marks today. With no ending, nothing marks it. Until this has a design, the heap machine covers less of Ochr than today's machine (H1).

7.9 Performance

Unmeasured. Reified computations allocate a closure per request; thunks and sharing may make re-normalisation cheaper than today's substitution sweeps. The checker is deliberately naive, so the bar is "not dramatically slower", and it must be measured on the suite before any port lands.

8 The bridge: two machines

This is the referees' strongest point about the design, and it decides the recommendation.

The paper's rules are about today's machine: borrows that move, loans, ending a borrow by substitution, resolution, owners found from loans. So is everything built on them: the well-formedness conditions on states, the agreement relation between the generic and the instance path in the paper's soundness theorem for its first-order fragment, and the `Pick` example that forces agreement to be stated after resolution. The lemmas of Section 4 are about the heap machine, where none of these exist. Nothing proves the two machines equivalent; their agreement is measured, on 1,166 runs of accepted functions in a slice that has no moves, no returned borrows and no typing.

So after a port, the checker would no longer implement the paper's rules, and a Lean proof about it would be about a machine the paper does not describe. The project's standing aim is that the paper's theorem is about the rules the checker runs. There are four ways to respond:

Option	What it takes	What it yields
(A) the heap machine becomes Ochr's semantics	rewrite the paper's machine rules, state conditions, resolution, the fragment theorem's agreement relation and the borrow rows of its table of side conditions, in heap terms; a new round of review; then port	no bridge: paper, checker and proofs about one machine, a simpler one (no resolution, no global substitution, worlds as rollback)
(B) keep today's rules; prove the machines equivalent	a simulation between the two machines, rule by rule, with a representation relation between their states	the bridge, at the price of a per-construct induction over two machines: the kind of proof the port was meant to avoid, doubled
(C) port the checker, leave the paper as it is	the port only	a Lean proof about a machine the paper does not describe; the correspondence tested, not proved. Not acceptable as the paper's theorem
(D) do not port	nothing	(P) stays measured on today's checker (193 of 193) and argued; consistency per program comes from certificates (docs/15), which need neither machine proved

Figure 3: Ways to answer the bridge question.

What closes the bridge is (A) or (B). (B) costs more than the port saves: it reintroduces exactly the per-construct reasoning over a large machine that motivated the design, and over two machines. (A) is the only option under which porting leaves a simpler system. But (A) is a decision about the calculus, not about the checker, and it should be taken on its own merits: is the heap machine a better operational semantics for Ochr than moving borrows? It has real advantages (no resolution, no substitution sweeps, worlds as rollback, references closer to Rust's model), and real gaps (Section 7.7, Section 7.8) that must be designed before anyone can say.

Why: Under (A) the port stops being a refactor of the checker and becomes the implementation of a new semantics. That reverses the order of work: the semantics has to exist, on paper and reviewed, before the code that implements it.

9 Costs and an order of work

Item	Estimate	Source
port the value layer to agreement on every verdict, node summaries and identity keys included	4–8 weeks	the stability work's estimate
renaming invariance of fresh handles	2–3 weeks	`heap-close` lane
thunks whose inputs are thunks, for (C)	about 1 week	`heap-close` lane
the evaluator never fabricates a handle	about 1 week	`frame-model` lane
(Sp) as fill and unfill of the scrutinee's node	about 1 week	`frame-model` lane
(M), typed and untyped runs agree	2–4 weeks, or free if typing is the same computation	`frame-model` lane
(Mono), monotone questions	1–2 weeks	my guess
design: returned borrows, exclusivity, worlds with forcing, types of unknowns (H1, H3–H5)	not estimated	
option (A): the paper's machine in heap terms, and its review	not estimated; my guess 2–4 weeks	
option (B): a simulation between the machines	more than any row above	`frame-model` lane

Figure 4: Costs, in weeks of focused agent work.

The estimates given add up to 12 to 20 weeks, before the design work and the paper. Under (A), with the paper, the total is of the order of four to six months. That buys the run half of stability. It does not touch the other gaps the referees found in the consistency plan: that truth is defined in a set model while every lemma speaks of machine values (the plan's semantic half), and that stability must be restated with an ordering.

Regardless of the port, three things are worth doing now, because each removes a way for a run to depend on something it should not, and none depends on the design: fix `EtaBoom` in today's checker, since it is an open closed proof of `False` on main (restore η-records when leaving an arm, or recompute their field types per world); merge `port-step1′`s borrow-holder index, which changed no verdict; and settle whether the generalisation records can be keyed by identity (H6).

If the port is pursued, the order is:

1. Design on paper H1 to H6 (Section 11), and write the heap machine's rules in the paper's style. My guess: 1 to 2 weeks.
2. Decide (A).
3. Port slice by slice, each step compared with today's checker on the whole suite: the data slice, then borrows with exclusivity, then case splits as fills with rolled-back worlds, then closing off as thunks, then returned borrows, then the typed path.
4. Prove, on the ported checker: renaming invariance, then (C) with nested thunks, (Sp), (Mono), and (M).

10 Recommendation

Do not port now. Settle the design questions on paper, write the heap machine's rules, and take the decision that actually matters: should the heap machine become Ochr's semantics (option A)? Port if and only if it should.

For porting. The design does what it was proposed to do. (P), locality and the frame part of closing off become lemmas about the interpreter, proved once, with no case of the machine inspected, where the direct route was estimated at 10,000 to 20,000 lines. Resolution, the global substitution of a refinement and the eager re-normalisation of sealed programs disappear. The slice agrees with today's checker on all 1,166 runs, all 579 forced thunks and all 193 checks of (P).

Against porting now.

1. **The design is incomplete exactly where it was attacked hardest.** Borrows returned by stuck calls have no design (Section 7.8); references are not yet exclusive (Section 7.7); the stated treatment of an unknown's type would rebuild `EtaBoom`, and the corrected one needs worlds that roll back forcing (Section 7.3); and the

steps that consult a type need a lemma (Mono) that is not free (Section 7.4). Each was found only by looking hard. Porting an incomplete design finds the rest as failing verdicts in the middle of the port, which is the most expensive time to find them.

2. **There is no bridge** (Section 8). Unless the paper adopts the heap machine, the port produces a checker the paper does not describe, and closing that gap by a simulation costs more than the port saves.
3. **It is not on the critical path to what is most at risk.** The referees' main finding about the consistency plan is in its semantic half, the gap between truth in the set model and the machine values every lemma speaks of; together with a Lean statement of stability that is false without an ordering, and an open proof of `False`. The port touches none of these.
4. **The guarantee users rely on already exists without it.** Every accepted program that certifies is consistent relative to Lean, whichever machine checked it.

My leaning on the real decision is weakly in favour of (A): the heap machine is the simpler semantics, and closer to how Rust borrows behave. But the question of returned borrows decides it, because that is where today's resolution earns its keep, and I would not commit before it has a design.

11 Open design questions, collected

Open design question H1 (borrows returned by stuck calls). How a stuck call that returns a borrow is closed off when there is no ending of borrows. A hole as a fresh cell read by thunks needs a rule for when the hole is final, since forcing is memoised; candidates are forcing only after the returned reference is dropped, or not memoising thunks that read a hole. Today's `Pick` example must come out right. Section 7.8.

Open design question H2 (refinements and fresh handles). Whether a refinement allocates its nodes from a namespace of its own, which keeps (P) free of renaming, and how renaming invariance is proved for the comparisons that need it anyway: forcing after other allocations, and relating an instance's heap to the generic heap filled in place. Section 7.5.

Open design question H3 (worlds and forcing). A world must roll back every write made inside it, forcings included, or a type forced under one arm's refinement leaks into a sibling arm, as in `EtaBoom`. What exactly a world may keep: today's γ_Γ-records only as shapes whose types are unforced, and the types of unknowns created inside the arm only while nothing kept names them. Section 7.3.

Open design question H4 (the type of an unknown). An unknown's type as a fixed handle to a type value that is itself refined and forced like any other value, and the questions about it (copy type, `Unit`) as monotone requests, so that (Mono) can be stated per request site. Section 7.3, Section 7.4.

Open design question H5 (exclusivity). A mark on each lent place, set by a borrow and checked and cleared by every access to the place, so that a stale reference is rejected as today. Its interaction with copy and restore, and with blocks that take places by borrow. Section 7.7.

Open design question H6 (identity keys). Whether keying the generalisation records, the conversion stack and the class cache by node identity loses only completeness, or reintroduces the failure that recognising re-derived sealed programs by their contents (D34) was made to fix. Section 7.2.

Open design question H7 (the bridge). Whether the heap machine becomes Ochr's semantics (option A), with the paper's rules and the fragment theorem restated in heap terms, or today's rules stay and the port is abandoned. Section 8.